MALLA REDDY UNIVERSITY

An Application Development Project Report

SECURE DIGITAL ASSETS AND LEGACY MANAGEMENT SYSTEM

Design & developed by

K.Geethika	2411CS040073
CH.Madhuri	2411CS040037
K.Theerthi	2411CS040074

GUIDED BY

Dr. M. Keerthi Priya

(Assistant Professor)

Department of Computer Science & Engineering (Cyber Security)

IIYR-IISEM

Malla Reddy University, Hyderabad

April-2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science & Engineering (Cyber Security)

CERTIFICATE

This is to certify that this is the Bonafide record of the application development project entitled **"SECURE DIGITAL ASSETS AND LEGACY MANAGEMENT SYSTEM"** submitted by **K.Geethika (2411CS040073), CH.Madhuri (2411CS040037), K.Theerthi (2411CS040074)** of B. Tech II year II semester, Department of CSE (Cyber Security) during the academic year 2025-26. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide Dr. M . Keerthi Priya
Assistant Professor

Dean (Cyber Security)
Dr. G. Anand Kumar
Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report entitled "Secure Digital Assets And Legacy Management System" has been carried out by us and submitted to the Department of Computer Science and Engineering (Cyber Security), Malla Reddy University, Hyderabad. This project work has been designed, developed & deployed by us, as part of the Application Development Project Work of this semester. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place:

Date:

K.Geethika	2411CS040073
CH.Madhuri	2411CS040037
K.Theerthi	2411CS040074

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude **to Dr. V. S. K Reddy, Vice Chancellor,** for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express my deepest appreciation to our project guide **Dr. T. Suneetha Assistant Professor** whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to our **PRC – convenor, Dr. G. Latha, , Associate professor** for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We thank our project coordinator, **Dr. M . Keerthi Priya,** for her timely support.

We are also grateful **to Dr. G. Anand Kumar, Dean Cyber Security**, for providing us with the necessary resources and facilities to carry out this project.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

K.Geethika 2411CS040073

CH.Madhuri 2411CS040037

K.Theerthi 2411CS040074

ABSTRACT

In the modern digital era, individuals store a significant amount of personal and sensitive information across various online platforms, including social media accounts, personal documents, financial records, and other digital assets. As the usage of digital services increases, managing and securing this information becomes a major concern, especially when users become inactive or are unable to access their accounts. Currently, there is no structured and reliable system that ensures secure handling and controlled access to such digital data, which can lead to difficulties in retrieving important information and risks related to unauthorized access or data loss. The Secure Digital Assets & Legacy Management System is a web-based application designed to address these challenges by providing a systematic and secure approach for managing digital legacy. The system enables users to store their digital asset information in a structured format and assign trusted nominees who can request access under predefined conditions. It ensures that access is granted only through a controlled and verified process, thereby maintaining confidentiality and preventing misuse of sensitive information. The application incorporates essential security features such as authentication mechanisms, structured data storage, and validation techniques to maintain data integrity and consistency. It also includes basic input validation and file handling features to ensure proper data management. The system follows a modular and layered architecture, which enhances scalability, maintainability, and ease of use. Overall, the proposed system provides a practical and efficient solution for managing digital assets in a secure environment, reducing complexity, improving accessibility, and ensuring that sensitive information is handled in a controlled and reliable manner while supporting future enhancements and real-world adaptability.

Keywords: Digital Assets, Legacy Management, Secure Access, Authentication, Data Security, Nominee Access, Access Control, Web Application

INDEX

CHAPTER 1

INTRODUCTION

In today's rapidly evolving digital world, individuals increasingly depend on online platforms to store and manage a wide range of personal and sensitive information. This includes social media accounts, email services, financial records, personal documents, and various other digital assets. With the continuous growth of internet-based services, the volume and importance of such digital data have significantly increased, making it a critical part of everyday life. As a result, ensuring the security, accessibility, and proper management of digital information has become a major concern.

Despite the widespread use of digital platforms, there is a lack of a unified and structured system that allows users to manage their digital assets in an organized and secure manner. Most existing systems focus only on storage and do not provide mechanisms for controlled access or proper management under specific conditions. This creates several challenges, including difficulty in tracking multiple assets, lack of centralized control, and increased risk of data loss or unauthorized access. Users often rely on unsafe practices such as storing credentials in unsecured formats or sharing login details, which further increases security risks.

The Secure Digital Assets & Legacy Management System is proposed as a solution to address these challenges by providing a structured and secure platform for managing digital information. The system enables users to store their digital assets in an organized format, assign trusted nominees, and define conditions under which access can be requested. It introduces a controlled access mechanism where requests are processed through verification before granting access, ensuring that sensitive data is handled responsibly..

Furthermore, the proposed system incorporates essential features such as user authentication, data validation, and structured data storage to maintain data integrity and security. It follows a modular and layered architecture, which improves system organization, scalability, and maintainability. The design focuses on providing a balance between security and usability, ensuring that users can manage their digital assets efficiently without compromising privacy.

Overall, this project aims to provide a practical and reliable approach to digital asset management by addressing the limitations of existing systems. It enhances security, improves accessibility, and introduces a structured method for managing digital legacy in a controlled and efficient environment, making it suitable for modern digital needs and future enhancements..

1.1 Problem Definition & Description

In the current digital era, individuals rely heavily on various online platforms to store and manage personal, professional, and financial information. This includes social media accounts, email services, banking details, digital documents, and other sensitive data. As the usage of digital services continues to grow, the volume and importance of such information have significantly increased. However, there is no centralized or structured system available to manage these digital assets in a secure and organized manner. Most users depend on multiple independent platforms, which makes it difficult to maintain proper control, accessibility, and security of their digital information.

One of the major issues arises when users become inactive or are unable to access their accounts due to unforeseen circumstances. In such cases, retrieving important data becomes highly difficult, as there is no standard mechanism to allow controlled access to trusted individuals. Users often resort to insecure methods such as sharing passwords or storing credentials in unsafe locations, which increases the risk of unauthorized access, data breaches, and misuse of sensitive information. This lack of proper management and security leads to both privacy concerns and potential loss of valuable digital assets..

A significant problem arises when users become inactive, forget credentials, or are unable to access their accounts due to unforeseen circumstances. In such situations, there is no proper mechanism for transferring access of these digital assets to trusted individuals. As a result, important data may be permanently lost or remain inaccessible, causing serious consequences such as financial loss, disruption of services, or emotional distress for family members. This lack of digital legacy planning has become a critical concern in modern digital life.

Additionally, existing systems and platforms do not provide integrated solutions that combine secure storage, nominee management, and controlled access. Most services focus only on data storage or account management, without addressing the need for secure transfer of ownership or access in case of inactivity or emergencies. This fragmented approach fails to provide a complete solution for digital asset management and legacy planning..

Moreover, there is a lack of proper verification and authorization mechanisms when granting access to sensitive data. Without structured validation processes, there is a high risk that unauthorized users may gain access to confidential information. This highlights the need for a system that ensures strong authentication, role-based access control, and verification before granting access to any digital asset.

1.2 Objectives of the Project

The primary objective of the VaultLegacy – Secure Digital Asset & Legacy Management System is to design and develop a secure and centralized platform that enables users to manage their digital assets efficiently. The system aims to provide a structured environment where users can store sensitive information such as account credentials, documents, and personal data in a safe and organized manner. By centralizing all digital assets, the project eliminates the need for scattered and insecure storage methods..

Another key objective is to implement a secure nominee management system that allows users to assign trusted individuals who can request access to digital assets under predefined conditions. The system ensures that access is not granted directly but follows a controlled process involving verification and approval. This helps in maintaining privacy while also enabling legitimate access when required, such as during inactivity or emergency situations..

The project aims to demonstrate a practical and scalable solution for digital legacy management that can be extended in the future. By using modern technologies and modular architecture, the system can be enhanced with advanced features such as automated inactivity detection, real-time alerts, and improved security measures. Overall, the objective is to create a reliable system that ensures both accessibility and security of digital assets.

1.3 Scope of the Project

The scope of the VaultLegacy – Secure Digital Asset & Legacy Management System focuses on providing a centralized platform for storing, managing, and securely accessing digital assets. The system is designed to allow users to store various types of sensitive information such as account credentials, personal documents, financial details, and private notes in an organized and structured manner. It ensures that all digital assets are maintained in a single system instead of being scattered across multiple insecure storage methods.

The project includes the implementation of a nominee management feature, where users can assign trusted individuals who are eligible to request access to their digital assets. The system defines controlled conditions such as inactivity or emergency situations under which access can be requested. This ensures that digital information is not lost and can be accessed by authorized individuals when required, while still maintaining strict security protocols.

Another important aspect within the scope is the implementation of authentication and access control mechanisms. The system supports role-based access for different types of users such as Owner, Nominee, and Admin. Each role is provided with specific permissions to ensure that data is accessed and managed only by authorized users. Verification processes are included to validate access requests before granting permission to sensitive data.

However, the project also includes basic security features such as input validation, file upload restrictions, and secure data handling practices to minimize risks associated with unauthorized access and data breaches. Additionally, the system provides dashboards and notifications to help users monitor activities such as asset updates, access requests, and approvals.

CHAPTER 2

SYSTEM ANALYSIS

1. Existing System

1. Google Drive / Cloud Storage Systems

Overview: Cloud storage platforms such as Google Drive are widely used for storing digital files and documents online. These systems allow users to upload, access, and share files from anywhere. However, they mainly focus on storage and sharing features and do not provide structured access control based on predefined conditions or nominee-based access mechanisms.

2. Password Managers (e.g., LastPass)

Overview: Password management systems like LastPass help users securely store and manage their login credentials. They use encryption to protect sensitive information and provide easy access through a master password. However, these systems are limited to credential management and do not support complete digital asset organization or controlled access workflows involving multiple roles.

3. Basic Account Recovery Systems

Overview: Many applications provide account recovery mechanisms such as email or OTP-based verification. These systems help users regain access to their accounts but do not offer a structured approach for controlled access sharing or long-term digital data management. They are limited to individual account recovery rather than complete asset handling.

4. Manual Data Sharing Methods

Overview: In many cases, users rely on manual methods such as sharing passwords or storing information in physical or unstructured digital formats. These methods are insecure and lack proper verification, making them vulnerable to unauthorized access and data misuse.

Existing systems such as cloud storage platforms, password managers, and account recovery systems mainly focus on individual functionalities like storage, security, or access recovery. They do not provide a unified system that integrates secure storage, nominee assignment, and controlled access mechanisms

.

In contrast, the Secure Digital Legacy Management System is designed as a structured platform that combines all these features into a single system. It provides secure data storage, role-based access control, verification processes, and controlled sharing of digital assets, making it more efficient and suitable for managing digital information in an organized way.

2.1 Background & Literature review

The Secure Digital Legacy Management System was developed with the idea of providing a centralized platform for storing digital assets and managing controlled access to them. The system focuses on secure data storage, nominee assignment, and verification-based access control using a simple and user-friendly interface . The goal of this project is to simplify the management of digital information while ensuring data security and privacy. By integrating authentication, storage, and access control into a single platform, the system provides an efficient solution that reduces complexity and improves usability for users without requiring advanced technical knowledge.

Literature Survey

- Digital Data Security and Access Control Systems

  Authors: M. Bishop, C. Gates
  Journal: IEEE Security & Privacy
  This study discusses different approaches for securing digital data and controlling access using authentication and encryption techniques. It highlights the importance of protecting sensitive data and ensuring that only authorized users can access it.

- A Web Application Security and Vulnerability Management

  Authors: S. Kals, E. Kirda, C. Kruegel, N. Jovanovic
  Journal: Proceedings of the World Wide Web Conference (WWW)
  This research focuses on common web application vulnerabilities such as SQL injection and cross-site scripting (XSS). It emphasizes secure design practices and validation techniques which are important for building secure web-based systems..

- Database Security and Data Management

  Authors: Elisa Bertino, Ravi Sandhu
  Journal: IEEE Transactions on Knowledge and Data Engineering
  This paper explains the importance of structured databases and secure data storage. It discusses how relational databases can be used to manage user information, assets,and access control effectively.

It highlights how relational database systems help in organizing data into tables with proper relationships, ensuring consistency and integrity. The study also discusses the use of access control mechanisms, such as role-based permissions, to restrict unauthorized access to data

- Authentication and Authorization Mechanisms

  Authors: R. Sandhu, D. Ferraiolo, R. Kuhn
  Journal: IEEE Computer
  This study focuses on user authentication methods and role-based access control systems. I
I it highlights how proper authentication and authorization help in preventing unauthorized access and improving system security.

- Cloud-Based Application Development

  Authors: Armbrust, Fox, Griffith, et al
  Journal: Communications of the ACM
  This paper discusses modern cloud-based technologies used in web application devdevelopment. It explains the use of backend services, APIs, and scalable infrastructure which are relevant to systems like Supabase and modern frontend frameworks.

2.1.2 Limitations of Existing System

Existing systems mainly focus on individual functionalities such as storage, password management, or account recovery, rather than providing a complete and integrated solution. These systems are designed to handle specific tasks, but they do not combine all required features into a single platform. As a result, users need to depend on multiple applications, which increases complexity and reduces efficiency in managing digital information.

Another major limitation is the lack of a structured mechanism for assigning authorized persons and managing controlled access to digital assets. Most existing systems do not support role-based access or nominee-based access features. This creates difficulties when controlled sharing of information is required, as there is no proper system to define who can access the data and under what conditions.

In addition, many systems lack proper verification-based access control mechanisms. Access is often granted through basic authentication methods without additional validation steps, which may lead to security risks or unauthorized access. Sensitive information may not be adequately protected, especially when there are no strong security layers such as multi-level verification or access monitoring.

Furthermore, users face challenges in organizing and managing their digital assets in a structured and secure manner. Data is often scattered across different platforms, making it difficult to maintain consistency and track important information. Manual methods of sharing data are also commonly used, which are insecure and do not provide proper protection, monitoring, or control over sensitive information.

2.2 Proposed System

The proposed system, Secure Digital Legacy Management System, is designed to provide a centralized platform for managing digital assets in a secure and structured manner. It integrates multiple functionalities such as data storage, user authentication, nominee assignment, and controlled access into a single system. This reduces the need for multiple applications and simplifies the overall management of digital information.

The system introduces a structured mechanism for assigning nominees and defining access conditions. Users can securely store their digital assets and specify authorized persons who can request access when required. Access is not given directly; instead, it follows a controlled process based on predefined conditions and verification steps, ensuring that sensitive data is protected at all times.

In addition, the proposed system implements verification-based access control to enhance security. It includes authentication mechanisms and role-based access management to prevent unauthorized access. Each request for access is verified before approval, which reduces security risks and ensures that only authorized users can view or manage the data.

Furthermore, the system provides a user-friendly interface that allows easy organization and management of digital assets. It ensures data consistency, privacy, and reliability while maintaining a simple workflow. Overall, the proposed system offers a more secure, efficient, and organized solution compared to existing systems, making it suitable for modern digital data management needs.

1. Advantages of the Proposed System

- **Centralized Security Monitoring**

  The system provides a unified platform to store and manage all digital assets in one place. This eliminates the need for multiple applications and allows users to easily organize, update, and access their information through a single interface.

- **Secure Storage of Digital Assets**

The system ensures that all digital assets are stored securely using proper data protection mechanisms. Sensitive information is safeguarded from unauthorized access, reducing the risk of data breaches.

- **Controlled Access Mechanism**

  Unlike traditional systems, this system provides a structured method for assigning nominees and defining access conditions. Access to data is granted only after proper verification, ensuring controlled and secure sharing of information.

- **Verification-Based Access Control**

  The system implements authentication and verification processes before granting access. This additional security layer helps in preventing unauthorized users from accessing sensitive data.

- **Role-Based Access Management**
  The system supports different roles such as user, nominee, and administrator. Each role has specific permissions, which improves security and ensures that users can only perform authorized actions.

- **User-Friendly Interface**

  The application is designed with a simple and easy-to-use interface. Users can manage their digital assets without requiring advanced technical knowledge, making the system accessible to a wide range of users.

- **Improved Data Organization**

  The system helps users maintain their data in a structured format. This improves data consistency, reduces confusion, and makes it easier to retrieve important information when required.

- **Enhanced Data Privacy and Security**

  By integrating authentication, access control, and secure storage, the system ensures high levels of data privacy. It minimizes risks associated with manual data handling and unsecured sharing methods.

3. Software & Hardware Requirements

1. Software Requirements

- Operating System: Windows / Linux
- Frontend: React.js
- Backend: Node.js
- UI Design: Tailwind CSS / Stitch
- IDE: Visual Studio Code
- Programming Languages: HTML, CSS, JavaScript

2.3.2 Hardware Requirements:

- Processor: Intel i3 or equivalent
- RAM: Minimum 4 GB
- Storage: 20 GB free disk space

4. Feasibility Study

1. Technical Feasibility

The proposed Secure Digital Legacy Management System is technically feasible as it is developed using widely used and well-supported web technologies. The frontend is built using React.js, while the backend is implemented using Node.js, both of which are efficient and scalable for web application development. The system uses modern UI design tools such as Tailwind CSS or Stitch to create a user-friendly interface. These technologies are compatible with each other and allow smooth communication between frontend and backend through APIs. The application can be deployed easily using platforms such as Vercel or other cloud-based hosting services, ensuring proper execution and accessibility.

Furthermore, the implementation of core functionalities such as user authentication, digital asset management, nominee assignment, and controlled access mechanisms is achievable using standard development practices. The system follows a structured workflow that includes login, data storage, request handling, and verification processes. These features are simple to implement and suitable for a college-level project without requiring complex algorithms or advanced infrastructure. The availability of documentation, libraries, and community support for all selected technologies further simplifies development. Overall, the project can be developed and tested using basic system resources, making it practical and achievable.

2.4.2 Robustness & Reliability

The proposed system is designed to be robust by ensuring stable performance during normal operations. The application follows a modular structure where frontend, backend, and storage functionalities are handled separately. This reduces the chances of system failure and allows each component to function independently. Proper validation techniques are used to handle user inputs and prevent errors, ensuring smooth execution of operations..

Reliability is maintained through consistent data handling and secure communication between system components. The system includes authentication and access control mechanisms to ensure that only authorized users can perform actions. Error handling techniques are implemented to manage unexpected situations without affecting system performance. Additionally, the use of standard frameworks ensures stability and predictable behavior of the application.

The system is also designed to handle multiple user interactions efficiently. Even if minor issues occur, the overall system continues to function without major disruption. The structured workflow and controlled access process ensure that data is handled securely and consistently. This makes the system dependable for managing digital assets and access control in a secure environment.

Even if minor issues such as temporary delays or input errors occur, the overall system continues to function without major disruption due to proper error handling and validation mechanisms

2.4.3 Economic Feasibility

The proposed Secure Digital Legacy Management System is economically feasible as it is developed using open-source and freely available technologies such as React.js, Node.js, and Tailwind CSS. These technologies do not require any licensing cost, which keeps the overall development cost very low. The system can be developed using standard development tools like Visual Studio Code, which is also free to use. Deployment can be done using cost-effective platforms such as Vercel or other cloud services, making it affordable for students and small-scale implementations without requiring significant financial investment.

The hardware requirements for the system are minimal, as it can run efficiently on a basic computer with moderate specifications. There is no need for high-end servers or expensive infrastructure, which further reduces the cost of implementation. Development and maintenance can be handled by a small team, avoiding the need for specialized or highly paid professionals. Since the system is designed for academic and controlled environments, operational expenses remain low compared to large-scale enterprise systems.

In comparison to existing systems that may require multiple tools or paid services for storage, security, and access management, the proposed system integrates all functionalities into a single platform. This reduces the need for additional applications and associated costs. The system also minimizes risks related to data mismanagement by providing a structured and secure approach, which indirectly helps in avoiding potential losses.

CHAPTER 3

ARCHITECTURAL DESIGN

1. Modules Design

The project Secure Digital Legacy Management System has mainly 7 modules to perform its functions which includes User Interface Module, Authentication & Security Module, Digital Asset Management Module, Nominee Management Module, Access Request Module, Verification & Access Control Module, and Notification & Monitoring Module.

1. User Interface Module

The Represents the frontend of the system where users interact with the application. This module provides a user-friendly interface for registration, login, adding digital assets, assigning nominees, and managing access requests. It is built using React and Vite with a Glassmorphism Dark Theme styling to ensure smooth navigation and proper display of all system functionalities.

2. Authentication & Security Module

Handles user registration and login processes. It ensures secure authentication using credentials and session management. This module protects the system from unauthorized access and maintains user identity throughout the application using secure token-based mechanisms.

3. Digital Asset Management Module

Allows users to add, update, and manage their digital assets such as account details, documents, passwords, and other important information. It ensures that all data is stored in a structured and secure manner accessible only to the authorized user.

4. Nominee Management Module

Enables users to assign and manage nominees who are authorized to request access to digital assets in case of an emergency or the user's absence. This module maintains nominee details and links them with the respective user data, ensuring a secure and structured nomination process..

5. Access Request Module

Allows nominees to raise an access request to view or retrieve the digital assets of the assigned user. This module handles the complete request lifecycle from submission to approval or rejection, ensuring that only legitimate nominees can initiate the process.

6. Verification & Access Control Module

Verifies the identity of the nominee before granting access to the user's digital assets. This module applies security checks and validation mechanisms to confirm the authenticity of the access request. Once verified, controlled access is granted to the nominee in a secure manner.

7. Notification & Monitoring Module

Manages all system notifications sent to users and nominees regarding access requests, approvals, and important updates. This module continuously monitors system activities and keeps both users and nominees informed about the status of their data and requests in real time.

2. Method & Algorithm Design

The project **"Secure Digital Assets and Legacy Management System"** uses different methods to manage user authentication, digital asset storage, nominee assignment, access requests, and administrative verification. The system ensures that digital information is stored securely and accessed only by authorized users.

User Authentication.

Algorithm-1: User_Authentication()

BEGIN

1. User opens the application login page.
    1. User enters login credentials (email and password).
    2. System validates the input fields
    3. Send authentication request to the server.
    4. Server checks whether the user exists in the database.

IF user exists AND password is correct
THEN

- Generate authenticated session
- Redirect user to Dashboard page

ELSE

- Display login error message
- Request user to enter valid credentials

5. End authentication process.

END

3.2.2 Digital Asset Management

User Authentication.

Algorithm-2: Digital_Asset_Management()

BEGIN

1. User logs into the system.
2. Navigate to Legacy Setup / Wallet section.
3. User selects option to add digital asset.
4. User enters asset details such as name, description, and related information.
5. System validates the input data.

IF data is valid
THEN

- Store asset information in database

ELSE

- Display validation error message

6. Display confirmation message to the user.

END

3. 2.3 Nominee Assignment

Algorithm-3: Nominee_Assignment()

BEGIN

1. User accesses the Legacy Setup module.
2. User selects option to add nominee.
3. Enter nominee details such as name, email, and relationship.
4. System verifies the entered information.

   IF details are valid

- Store nominee information in database

   ELSE

- Display error message

6. Confirm nominee assignment.

END

3.2.4 Claim Request Processing

Algorithm-4: Claim_Request_Process()

BEGIN

1. Nominee accesses the Claim Portal.
2. Nominee submits request to access digital assets.
3. System records request information in database.
4. Request is forwarded to Admin module.
5. Admin reviews the claim request.

   IF request approved

- Grant access to digital assets

   ELSE

- Reject request and notify nominee

6. Update claim request status.

END

3.2.5 Admin Verification

The Admin Verification module ensures that access requests are reviewed and approved only by authorized administrators before granting access to digital assets. This process helps maintain security, prevents unauthorized access, and ensures that only validated requests are processed. The system evaluates request details and allows the administrator to approve or reject the request based on verification results.

Algorithm: -

Algorithm: Admin_Verification()

BEGIN

1. Initialize Verification Configuration:
   a) Request_Status = Pending
   b) Admin_Access_Level = Authorized
   c) Verification_Log = Enabled

2. For each request in Access_Request_Database:

Retrieve Request Details:

a) request_id = request.request_id

b) requester_name = request.requester_name

c) nominee_id = request.nominee_id

d) asset_owner_id = request.asset_owner_id

e) request_time = request.timestamp

f) request_status = request.status

3. Verification Check:

IF request_status == "Pending“

a) Display request details to Admin
b) Admin reviews request information

4. Admin Decision Process:

IF Admin selects **Approve**

IF approved

Grant controlled access to digital assets

ELSE

Deny access request

Update request status in database.
Notify user about decision.

6. END

3. Project Architecture

1. Architectural Diagram

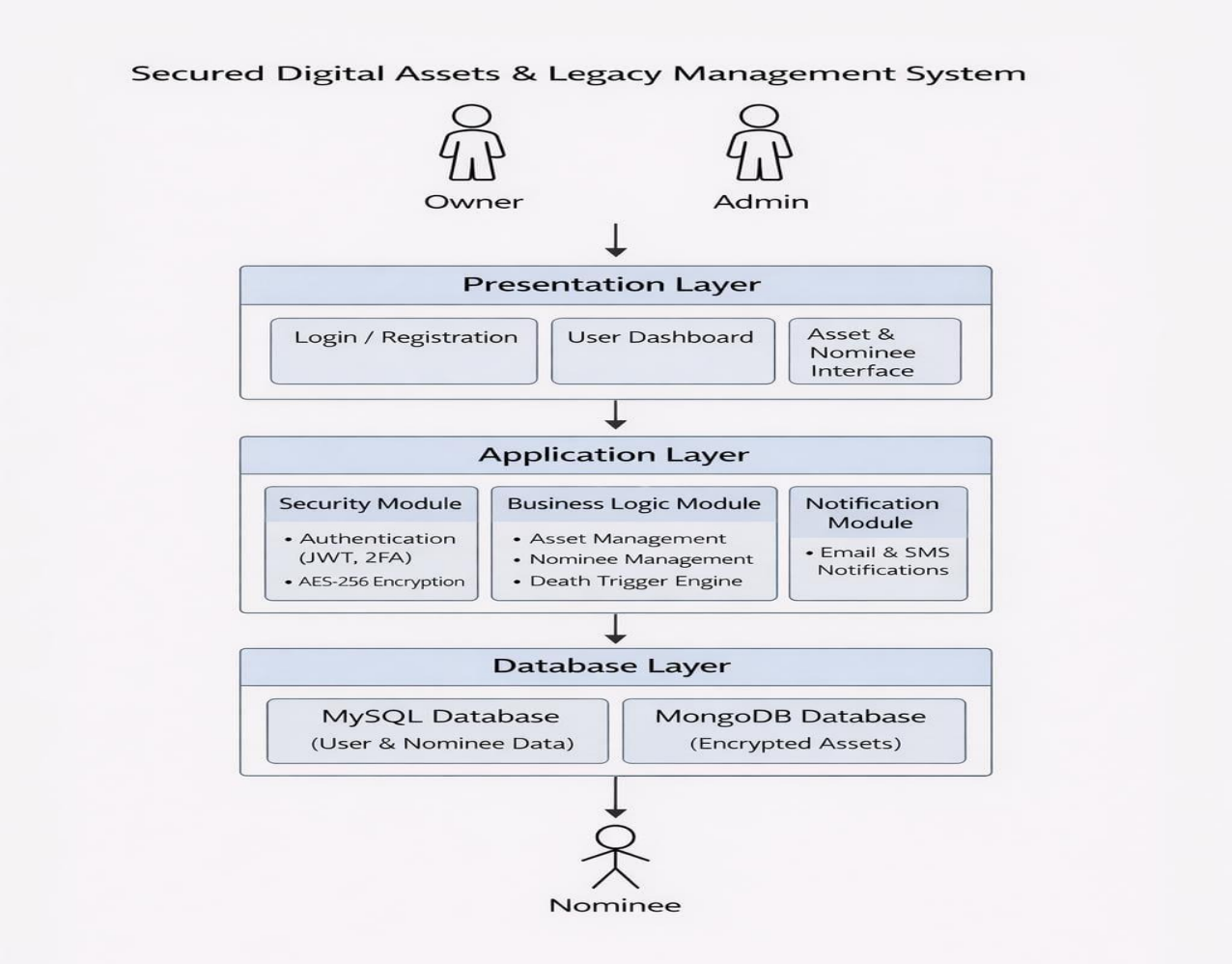


Figure 1: System functioning structure

The Secure Digital Assets and Legacy Management System is designed using a layered web architecture to ensure better organization, security, and system performance. It consists of three main layers: Presentation Layer, Application Layer, and Database Layer, each handling specific system operations..

The Presentation Layer provides the user interface for interaction through modules like Login, Dashboard, Legacy Setup, Wallet Management, Claim Portal, and Admin Vault. It allows users, nominees, and administrators to access and use system features in a simple and efficient way.

The Application Layer processes core system functionalities such as authentication, asset management, nominee handling, and claim verification, while the Database Layer stores all system data in a structured format for secure storage and easy retrieval.

3.3.2 Data Flow Diagram

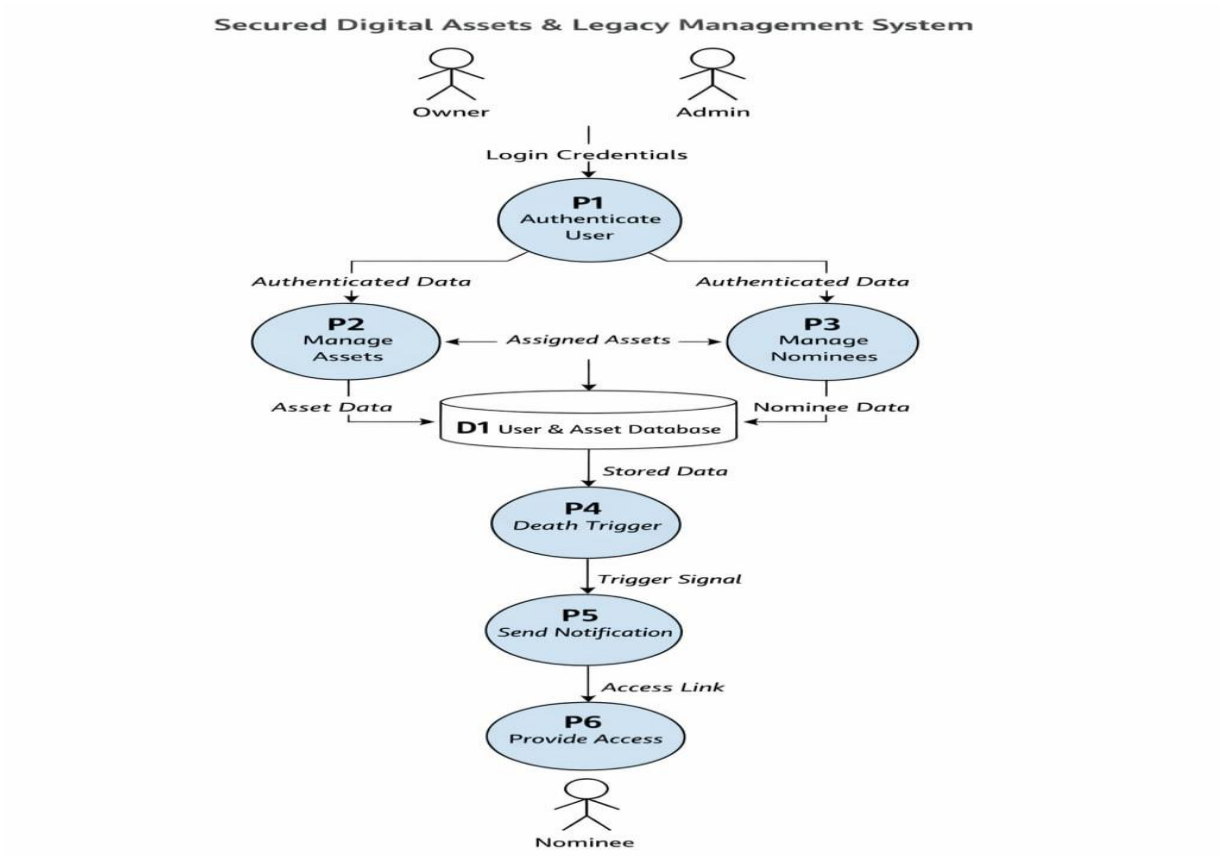


Figure 2: Data Flow diagram of the logs and data

The Data Flow Diagram (DFD) of the Secure Digital Assets and Legacy Management System illustrates how data flows between users, system processes, and the database. Users interact with the system through the web interface by providing inputs such as login credentials, digital asset details, nominee information, and claim requests. These inputs are sent to the application layer, where they are validated and processed according to system logic. The processed data is then stored in or retrieved from the database as required. The system generates appropriate outputs, such as authentication results, updated asset information, and claim status, which are displayed back to the user. This structured flow ensures secure, efficient, and controlled data handling throughout the application..

3.3.3 Class Diagram

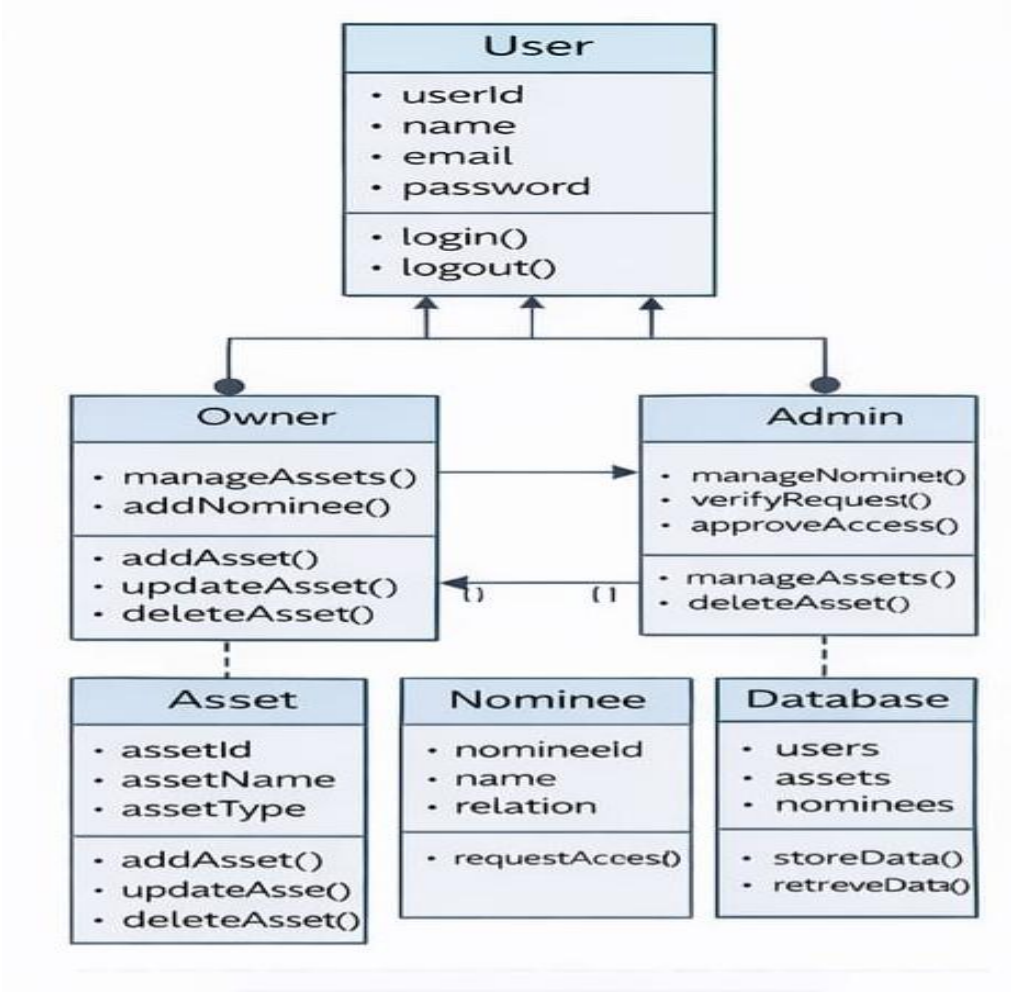


Figure 3: Privileges and functions data diagram of each node in the project

The Class Diagram of the Secure Digital Assets and Legacy Management System represents the structure of the system by showing different classes, their attributes, and relationships. The main classes include User, Digital Asset, Nominee, Claim Request, and Admin. The User class manages login and asset-related operations, while the Digital Asset class stores information about user assets. The Nominee class is associated with the user for access requests, and the Claim Request class handles the process of requesting access to assets. The Admin class is responsible for verifying and approving requests. The relationships between these classes ensure proper data organization and system functionality.

3.3.4 Use Case Diagram

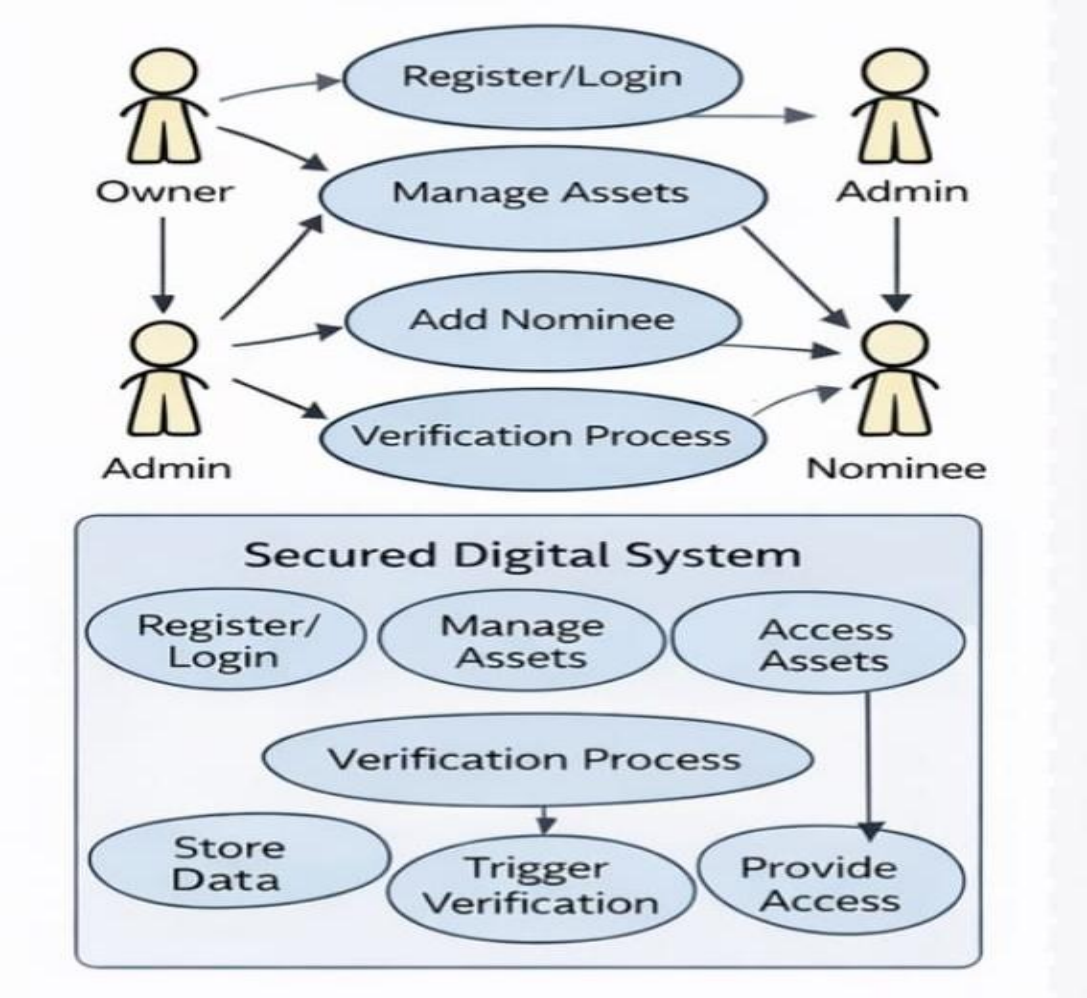


Figure 4: Uses Data diagram of each users and system

The Use Case Diagram illustrates the interactions between different actors and the system functionalities. The primary actors in the system are User, Nominee, and Admin. The User can perform actions such as login, manage digital assets, assign nominees, and manage wallet information. The Nominee can submit claim requests to access assets. The Admin is responsible for verifying requests and managing system operations. This diagram helps to clearly represent how each actor interacts with the system and the functionalities they can perform..

3.3.5 Sequence Diagram

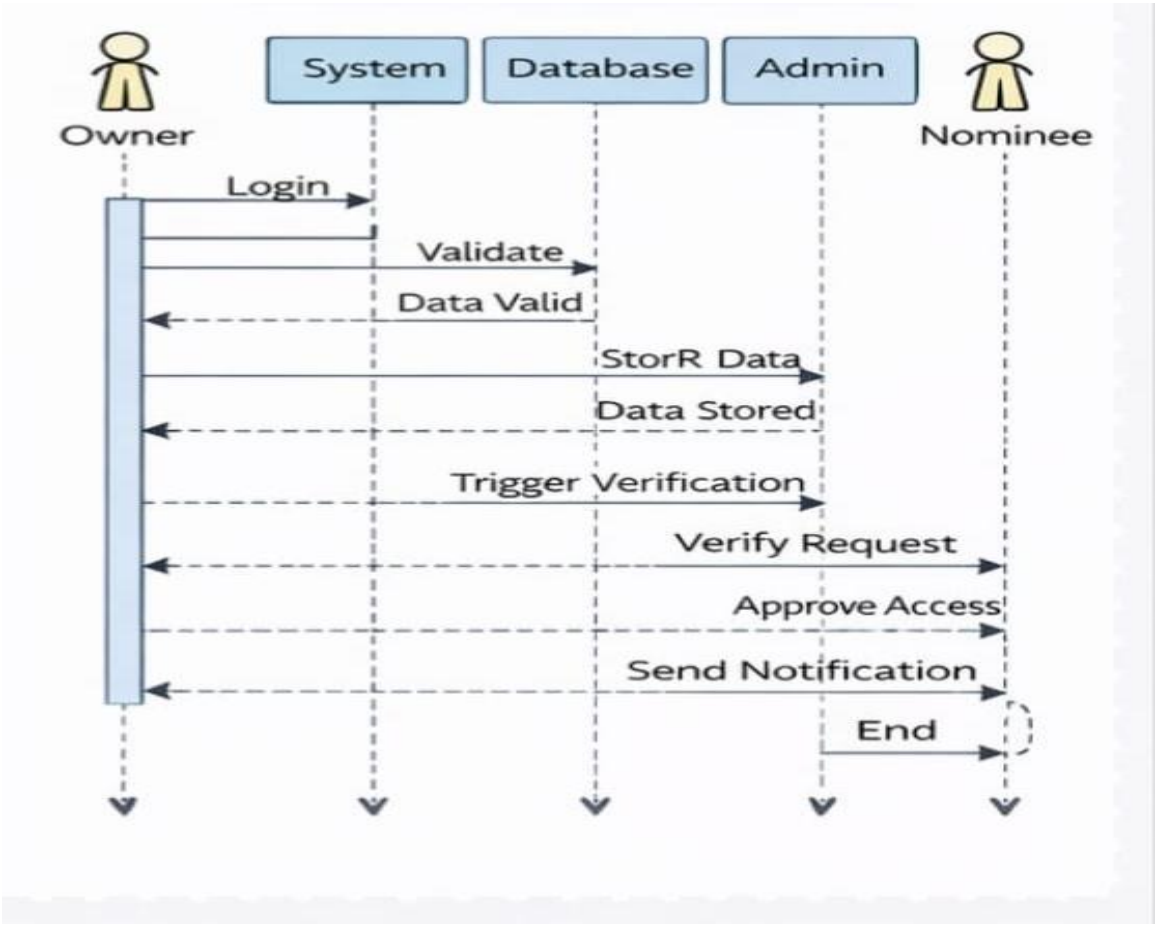


Figure 5: Response sequence diagram

The sequence diagram illustrates the step-by-step interaction between different components of the Secure Digital Assets and Legacy Management System during a typical operation. When a user accesses the system, they first interact with the login interface by entering their credentials. The request is sent to the application layer, where authentication is performed by verifying the user data from the database. After successful login, the user can access the dashboard and perform actions such as managing digital assets, assigning nominees, or initiating transfer requests.

When a nominee submits a claim request through the Claim Portal, the request is processed by the application layer and stored in the database. This request is then forwarded to the Admin module for verification. The admin reviews the request and decides whether to approve or reject it. Based on the admin's decision, the system updates the request status in the database and sends the response back to the user or nominee. This sequence ensures secure communication and controlled access throughout the system.

3.3.6 Activity Diagram

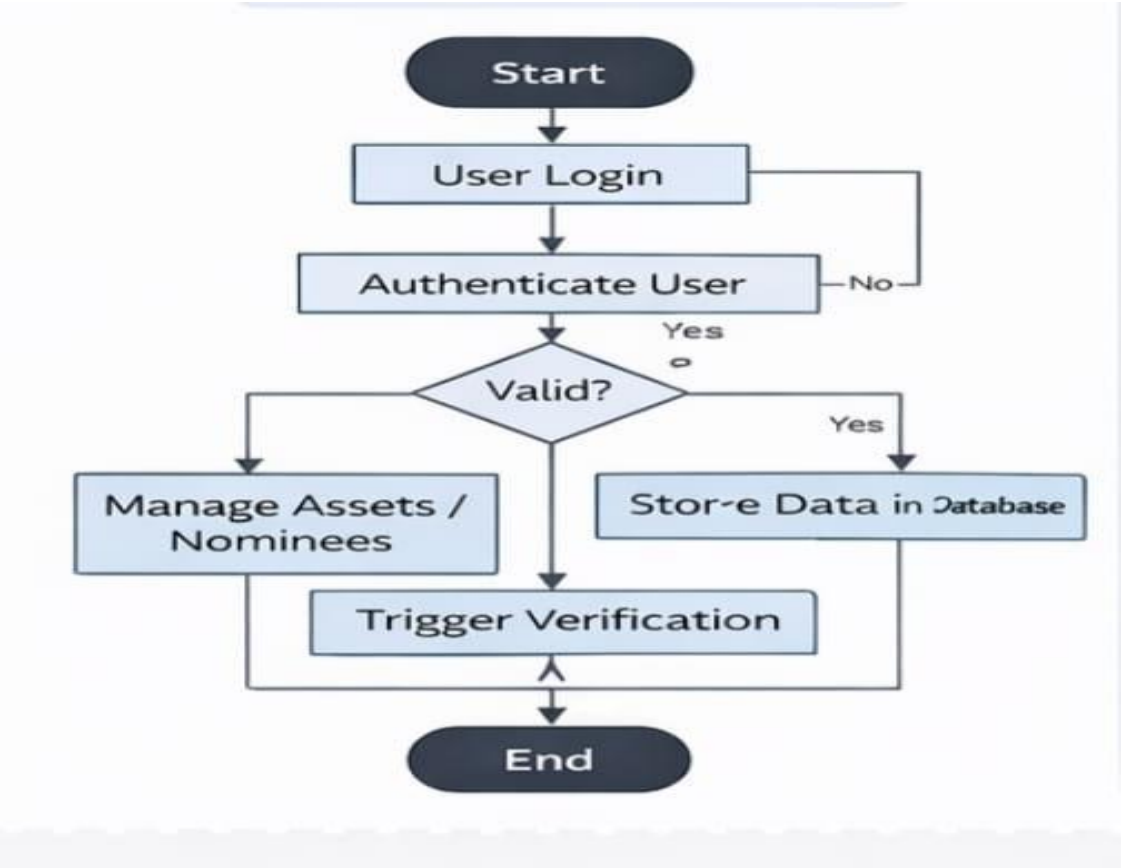


Figure 6: Activity process diagram

The Activity Diagram represents the workflow of the Secure Digital Assets and Legacy Management System by showing the sequence of activities performed by users and the system. The process begins with the user logging into the system, followed by authentication and access to the dashboard. The user can then perform various activities such as managing digital assets, assigning nominees, and handling wallet or transfer operations. If a nominee submits a claim request, the system processes the request and forwards it to the admin for verification. Based on the admin's decision, the request is either approved or rejected, and the system updates the status accordingly. The process ends with the display of results or confirmation messages, ensuring smooth and controlled system operation.

3.3.7 Deployment diagram

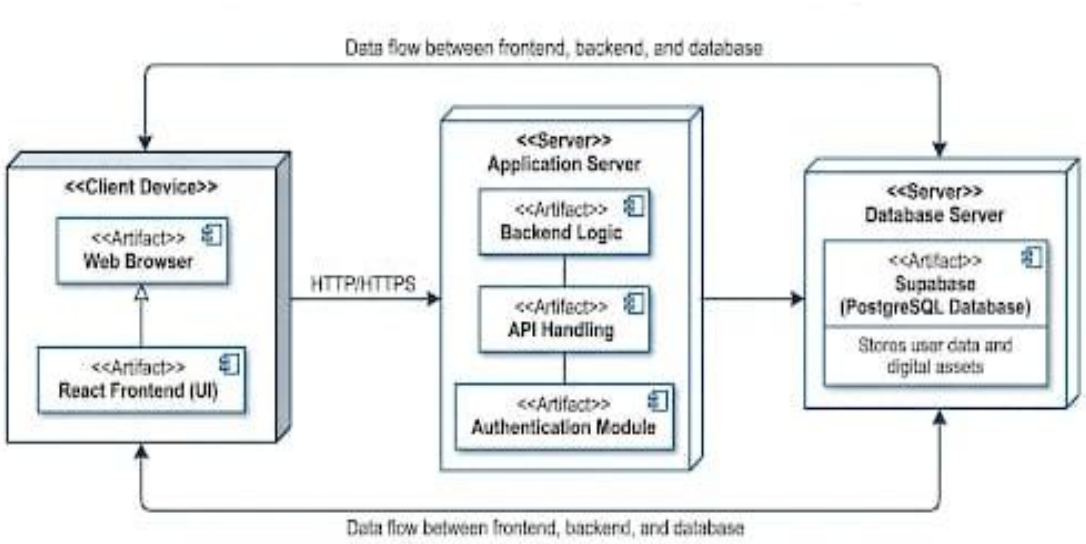


Figure: Deployment Diagram

Figure 7: Software and platforms required diagram

The Deployment Diagram represents the physical structure of the Secure Digital Assets and Legacy Management System and shows how different components of the application are deployed across hardware and software environments. The system follows a client-server architecture where the user interacts with the application through a web browser.

The frontend application is deployed on a web hosting platform and is accessed by users through client devices such as laptops or mobile devices. The backend services handle authentication, asset management, claim processing, and verification logic. The database server stores all system data including user information, digital assets, nominee details, and claim requests.

The communication between the client, server, and database is carried out through secure network connections. This deployment structure ensures efficient system performance, data security, and easy scalability of the application.

.

CHAPTER 4

IMPLEMENTATION & TESTING

1. Coding Blocks

1. Main Functional block:-

```
 import { createClient } from '@supabase/supabase-js';

// 1. Securely load Environment Variables from .env file
// Using import.meta.env for Vite-based React applications
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

// 2. Validate environment configuration before initialization
if (!supabaseUrl || !supabaseKey) {
  console.error('Core System Error: Missing Supabase Environment Variables.');
  console.warn('Ensure VITE_SUPABASE_URL and
VITE_SUPABASE_ANON_KEY are correctly set.');
}

// 3. Initialize the Core Database Interface with Advanced Settings
export const supabase = createClient(supabaseUrl, supabaseKey, {
  auth: {
    autoRefreshToken: true,     // Automatically refreshes expired JWT tokens
    persistSession: true,       // Keeps user logged in across browser restarts
    detectSessionInUrl: true,   // Required for password resets and magic link logins
    storageKey: 'valet-one-auth-token', // Custom local storage key for security
  },
  db: {
    schema: 'public',           // Targets the primary secure database schema
  }
});
```

```
import { Routes, Route, Navigate } from 'react-router-dom';
import Layout from './components/Layout';
import ProtectedRoute from './components/ProtectedRoute';


// Import Pages
import Login from './pages/Login';
import ClaimPortal from './pages/ClaimPortal';
import Dashboard from './pages/Dashboard';
import LegacySetup from './pages/LegacySetup';
import AdminVault from './pages/AdminVault';

export default function App() {
  return (
    <Routes>
      {/* Public Pages */}
      <Route path="/login" element={<Login />} />
      <Route path="/claim-portal" element={<ClaimPortal />}
/>
       {/* Protected Owner Pages with Dashboard Layout */}
      <Route element={<ProtectedRoute />}>
        <Route element={<Layout />}>
          <Route path="/dashboard" element={<Dashboard />}
/>
          <Route path="/legacy-setup" element={<LegacySetup
/>} />
        </Route>
      </Route>

      {/* Secure Administrator Page */}
      <Route
        path="/admin-vault"
        element={<ProtectedRoute
requireAdmin={true}><AdminVault /></ProtectedRoute>}
      />

{/* Fallback routing */}
      <Route path="*" element={<Navigate to="/login" />} />
    </Routes>
  );
}
```

```
import React, { useState } from 'react';
import { supabase } from '../supabaseClient';
import { useNavigate } from 'react-router-dom';

export default function SignUp() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [errorMsg, setErrorMsg] = useState(null);
  const navigate = useNavigate();

  const handleRegister = async (e) => {
    e.preventDefault();
    setLoading(true);
    setErrorMsg(null);

    try {
      // Create user identity in Supabase Auth
      const { user, error } = await supabase.auth.signUp({
        email: email,
        password: password,
      });

      if (error) throw new Error(error.message);

      alert("Registration Successful! Please check your email to verify.");
      navigate('/login');

    } catch (error) {
      setErrorMsg(error.message);
      console.error("SignUp Error:", error);
    } finally {
      setLoading(false);
    }
  };
```

```
return (
  <div className="flex justify-center items-center h-screen bg-gray-900">
    <form onSubmit={handleRegister} className="bg-gray-800 p-8 rounded-lg shadow-xl w-96">
      <h2 className="text-2xl text-white font-bold mb-6">Create Vault Account</h2>
      {errorMsg && <p className="text-red-500 mb-4">{errorMsg}</p>}
    <label className="block text-gray-300 mb-2">Email Address</label>
      <input
        type="email"
        className="w-full p-2 mb-4 rounded bg-gray-700 text-white"
        value={email} onChange={(e) => setEmail(e.target.value)} required
      />
  <label className="block text-gray-300 mb-2">Secure Password</label>
      <input
        type="password"
        className="w-full p-2 mb-6 rounded bg-gray-700 text-white"
        value={password} onChange={(e) => setPassword(e.target.value)} required
      />

      <button
        type="submit" disabled={loading}
        className="w-full bg-blue-600 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded"
      >
        {loading ? 'Registering...' : 'Sign Up'}
      </button>
    </form>
  </div>
 );
}
```

```
import React, { useState } from 'react';
import { supabase } from '../supabaseClient';
\ import { useNavigate } from 'react-router-dom';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isAuthenticating, setIsAuthenticating] = useState(false);
  const navigate = useNavigate();

  const proceedToVault = async (e) => {
    e.preventDefault();
    setIsAuthenticating(true);

    try {
      // Validate credentials against encrypted database
      const { data, error } = await supabase.auth.signInWithPassword({
        email: email,
        password: password,
      });

      if (error) {
        alert("Authentication failed: Invalid email or password.");
        return;
      }

      // Route logged-in user to their private legacy dashboard
      navigate('/dashboard');

    } catch (error) {
      console.error("Critical Login Error:", error);
    } finally {
      setIsAuthenticating(false);
    }
  };
```

```
 return (
   <div className="min-h-screen flex items-center justify-center bg-gray-950">
    <div className="max-w-md w-full p-6 bg-gray-900 border border-gray-700 rounded-lg shadow-2xl">
      <div className="text-center mb-8">
       <h1 className="text-3xl font-extrabold text-white">Valet One</h1>
       <p className="text-gray-400 mt-2">Secure Digital Legacy Portal</p>
      </div>

      <form onSubmit={proceedToVault}>
       <div className="mb-4">
        <input
         type="email" placeholder="Vault Email" required
         className="w-full px-4 py-3 bg-gray-800 text-gray-100 border border-gray-600 rounded focus:border-blue-500 outline-none"
         onChange={(e) => setEmail(e.target.value)}
        />
       </div>
       <div className="mb-6">
        <input
         type="password" placeholder="Passphrase" required
         className="w-full px-4 py-3 bg-gray-800 text-gray-100 border border-gray-600 rounded focus:border-blue-500 outline-none"
         onChange={(e) => setPassword(e.target.value)}
        />
       </div>
       <button
        type="submit"
        className="w-full py-3 bg-blue-600 text-white font-semibold rounded hover:bg-blue-500 transition duration-200"
       >
        {isAuthenticating ? 'Decrypting Vault...' : 'Access Vault'}
       </button>
      </form>
    </div>
   </div>
  );
}
```

```
import React, { useState } from 'react';
import { supabase } from '../supabaseClient';

export default function AddAssetForm({ refreshData, closeModal }) {
  const [assetName, setAssetName] = useState('');
  const [assetValue, setAssetValue] = useState('');
  const [isSaving, setIsSaving] = useState(false);

  const saveToVault = async (e) => {
    e.preventDefault();
    setIsSaving(true);

    try {
      // Ensure user is authenticated before insert
      const { data: { user } } = await supabase.auth.getUser();

      // Execute database insertion
      const { error } = await supabase
        .from('assets')
        .insert([
          {
            owner_id: user.id,
            asset_name: assetName,
            asset_value: assetValue
          }
        ]);

      if (error) throw error;

      alert("Asset securely stored in the Valet One Database.");
      refreshData(); // Triggers the parent dashboard to reload
      closeModal();  // Closes the popup UI

    } catch (err) {
      console.error("Insertion failed: ", err);
      alert("Failed to insert asset into database.");
    } finally {
      setIsSaving(false);
    }
  };
```

```
 return (
   <div className="fixed inset-0 bg-black bg-opacity-70 flex justify-center items-center z-50">
     <div className="bg-gray-800 p-8 rounded-xl shadow-2xl w-full max-w-md border border-gray-700">
       <h3 className="text-2xl text-white font-semibold mb-6">Inject Digital Asset</h3>
       <form onSubmit={saveToVault}>
         <div className="mb-4">
           <label className="text-sm font-medium text-gray-400 block mb-2">Asset Name / Type</label>
           <input
             type="text" className="w-full bg-gray-900 border border-gray-600 p-3 rounded text-white"
             placeholder="e.g., Bitcoin Wallet, Chase Bank" required
             onChange={e => setAssetName(e.target.value)}
           />
         </div>
         <div className="mb-6">
           <label className="text-sm font-medium text-gray-400 block mb-2">Estimated Value ($)</label>
           <input
             type="number" className="w-full bg-gray-900 border border-gray-600 p-3 rounded text-white"
             placeholder="50000" required
             onChange={e => setAssetValue(e.target.value)}
           />
         </div>
         <div className="flex justify-end space-x-3">
           <button type="button" onClick={closeModal} className="px-4 py-2 text-gray-400 hover:text-white">Cancel</button>
           <button type="submit" disabled={isSaving} className="px-6 py-2 bg-blue-600 text-white rounded font-medium">
             {isSaving ? 'Encrypting...' : 'Save Asset'}
           </button>
         </div>
       </form>
     </div>
   </div>
 );
}
```

```
import React, { useEffect, useState } from 'react';
import { supabase } from '../supabaseClient';

export default function NomineeDashboard({ user }) {
  const [claimStatus, setClaimStatus] = useState('Pending Review');
  const [inheritedAssets, setInheritedAssets] = useState([]);

  useEffect(() => {
    checkInheritanceStatus();
  }, []);

  const checkInheritanceStatus = async () => {
    // 1. Check if the admin approved the death certificate claim
    const { data: claimData } = await supabase
      .from('claims')
      .select('status, original_owner_id')
      .eq('nominee_email', user.email)
      .single();

    if (claimData) setClaimStatus(claimData.status);

    // 2. If Approved, calculate inherited wealth based on legacy
percentage
    if (claimData?.status === 'Approved') {
      const { data: assets } = await supabase
        .from('assets')
        .select('*')
        .eq('owner_id', claimData.original_owner_id);

      const { data: legacyData } = await supabase
        .from('nominees')
        .select('allocation_percentage')
        .eq('nominee_email', user.email)
        .single();
```

```
 const percentage = legacyData ? legacyData.allocation_percentage : 0;

     // Map final values
     setInheritedAssets(assets.map(asset => ({
       name: asset.asset_name,
       inherited_value: (asset.asset_value * (percentage / 100)).toFixed(2)
     })));
   }
  };

  return (
    <div className="p-8 bg-gray-900 border-t-2 border-blue-500 rounded p-
6 shadow-xl text-white">
     <h2 className="text-2xl font-bold">Secure Beneficiary Matrix</h2>
     <p className="mb-4">Claim Status: {claimStatus}</p>

     {claimStatus === 'Approved' && inheritedAssets.map((asset, index) =>
(
       <div key={index} className="bg-gray-800 p-4 mb-2 rounded border
border-gray-700">
         <h4 className="font-bold">{asset.name}</h4>
         <p  className="text-green-500  font-semibold">Inherited  Amount:
${asset.inherited_value}</p>
       </div>
     ))}
    </div>
  );
}
```

```
import React, { useState } from 'react';
import { supabase } from '../supabaseClient';

export default function AdminVerification({ claim, close, refreshDashboard })
{
  const [isApproving, setIsApproving] = useState(false);

  // Generates the secure signed URL to view the uploaded Death Certificate
  const openCertificateStorage = () => {
    const { data } = supabase.storage
      .from('death_certificates')
      .getPublicUrl(claim.document_path);

    window.open(data.publicUrl, '_blank'); // Opens document securely in new
tab
  };

  const executeLegacyTransfer = async () => {
    setIsApproving(true);

    // Change Database status to 'Approved' to trigger the automated wealth
unlocking
    const { error } = await supabase
      .from('claims')
      .update({ status: 'Approved', resolved_at: new Date() })
      .eq('id', claim.id);

    if (error) {
      alert("Database error. Execution failed.");
    } else {
      alert("Verification successful. Assets are now unlocking for the
beneficiary.");
      refreshDashboard();
      close();
    }

    setIsApproving(false);
  };
```

```
 return (
   <div className="bg-gray-800 border border-red-900 p-6 rounded shadow-2xl text-white">
    <h2 className="text-xl font-bold text-red-500 mb-4">System Verification Control</h2>

    <p className="mb-2">Claimant Requesting Access: <span className="text-blue-400">{claim.nominee_email}</span></p>

    <button onClick={openCertificateStorage} className="w-full py-2 bg-gray-700 hover:bg-gray-600 rounded mb-6 text-sm">
      View Attached Source Document (PDF)
    </button>

    <div className="flex justify-between space-x-4 border-t border-gray-700 pt-4">
      <button onClick={close} className="px-4 py-2 bg-gray-600 text-sm rounded text-white hover:bg-gray-500">
        Cancel Overrride
      </button>
      <button
       onClick={executeLegacyTransfer} disabled={isApproving}
       className="px-6 py-2 bg-red-600 text-white font-bold rounded shadow-lg shadow-red-900/50 hover:bg-red-500"
      >
       {isApproving ? 'Executing...' : 'Authorize Vault Transfer'}
      </button>
    </div>
   </div>
  );
}
```

2. Test Case Study

1. Test Case 1

Test objective: To verify a new user can successfully register and log into the application.

Test Scenario: A new user is trying to create an account and authenticate themselves to access the Valet One dashboard.

Test Steps:

1. Open any browser to connect to internet.
2. Go to website Valet One web application.
3. Open the registration/login form.
4. In the Email or Input a new valid email address and secure password and submit the form query.

Expected Result: The user authentication pages load smoothly and successfully process the registration and login request..

Actual Result: The new user account is successfully created in the Supabase backend, and the user is redirected to the personalized Wallet/Dashboard page

Status: Pass

2. Test Case 2

Test objective: To verify that an authenticated account owner can add a new digital asset to their portfolio.

Test Scenario: An account owner is trying to log a new financial holding (e.g., a Bitcoin wallet) into their Valet One secure digital wallet.

Test Steps:

1. Open any browser to connect to internet.
2. Go to Valet One web application and log in.
3. Navigate to the main Dashboard or Wallets page.
4. Act Click on the "Add Asset" button and input valid asset details (Name, Value, Type).

Expected Result: The application smoothly opens the asset entry form and accepts the details without errors

Actual Result: he newly added asset is successfully displayed in the user's dashboard UI and saved to the Supabase database.

Status: Pass

3. Test Case 3

Test objective: To verify that an account owner can add a nominee and allocate a percentage of their assets

Test Scenario: A user wants to safely assign a beneficiary (nominee) and allocate a portion of their wealth in the event of their passing..

Test Steps:

1. Open any browser to connect to internet.
2. Go to Valet One web application and log in..
3. Navigate to the "Legacy Setup" module.
4. Input the Nominee details and specify a percentage share, then click save.

Expected Result: The Legacy Setup module loads properly and the system allows for form submission..

Actual Result: The Legacy Setup module loads properly and the system allows for form submission..

Status: Pass

4. Test Case 4

Test Objective: To verify that a registered nominee can successfully submit a death certificate to trigger a claim

Test Scenario: A registered nominee is attempting to notify the system of the owner's death by providing the official document.

.**Test Steps:**

1. Open the Dashboard website through the browser and then and then login using your credentials.
2. Go to the public-facing Valet One Claim Portal.
3. Go to Input the registered nominee email address.
4. Upload a digital copy of the Death Certificate and submit the claim.

Expected Result: The Claim Portal processes the file upload and successfully receives the claim submission..

Actual Result: The system records the claim securely in the database, sets its status to "Pending Review", and displays a success message..

Status:PASS

5. Test Case 5

Test objective: To verify that an Admin can review a pending claim.

Test Scenario: An administrator is navigating to the restricted vault to verify an uploaded death certificate and release the locked assets..

Test Steps:

1. Open the browser and connect to internet.
2. Consider a Go to the Valet One web application and log in with Admin priviliages
3. Navigate to the restricted "Admin Vault".
4. Open the pending claim, review the uploaded certificate document, and click ok approve button

Expected Result: The Admin dashboard loads smoothly and the uploaded document is visible for the administrator.

Actual Result: The claim status is successfully updated to "Approved", and the system triggers the predefined legacy wealth transfer..

Status: Pass

4.2.6 Test Case 6

Test objective: To validate that the system prevents total asset allocations from exceeding 100%.

Test Scenario: A user is attempting to assign more than 100% of their total wealth by setting up multiple nominees with high proportions..

Test Steps:

1. Open the Website from the browser and then give your credentials.
2. Go to the Valet One web application and log in.
3. Navigate to the "Legacy Setup" page where one nominee is already set to a 60% share
4. Attempt to add a second nominee with an allocation of 50% and submit.

Expected Result: The system prevents the form from submitting since the total allocation would reach 110%..

Actual Result: The application successfully identifies the incorrect total, catches the error, displays an error message, and does not save the invalid data..

Status: Pass

CHAPTER 5

RESULTS

1. Resulting Screens

1. Screen Shot 1

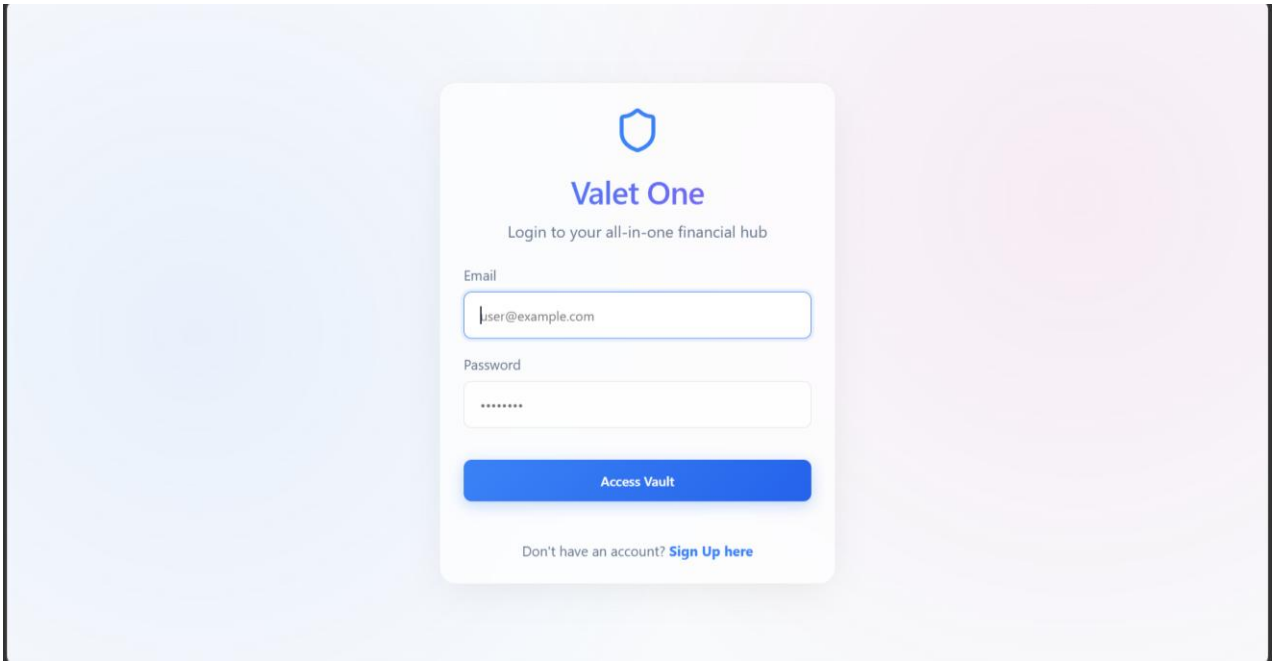


Figure 8: SIGN UP PAGE showing Valet One

User successfully enters valid credentials and logs into the system. The application authenticates the user and redirects to the dashboard, confirming successful login functionality.

5.1.2 Screenshot 2

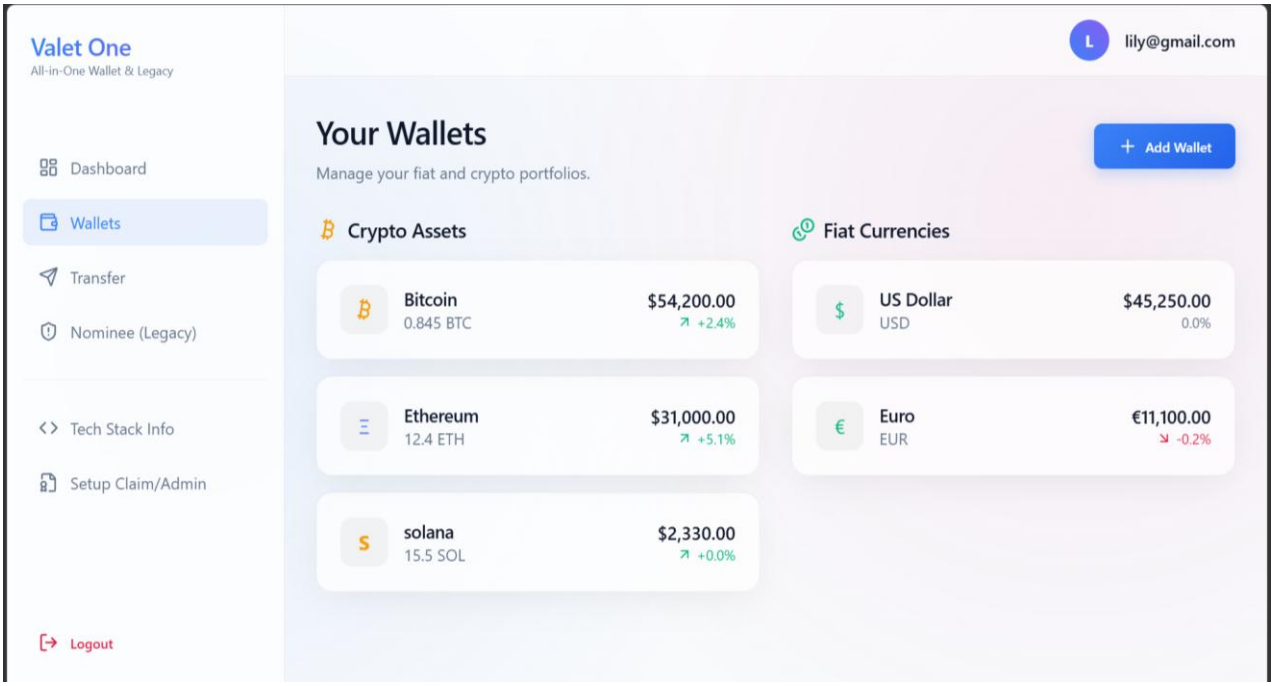


Figure 9: “Add Asset” form OR Dashboard showing added asset

The user inputs asset details such as name, type, and value. The system stores the asset successfully in the database and displays it on the dashboard.

5.1.3 Screenshot 3

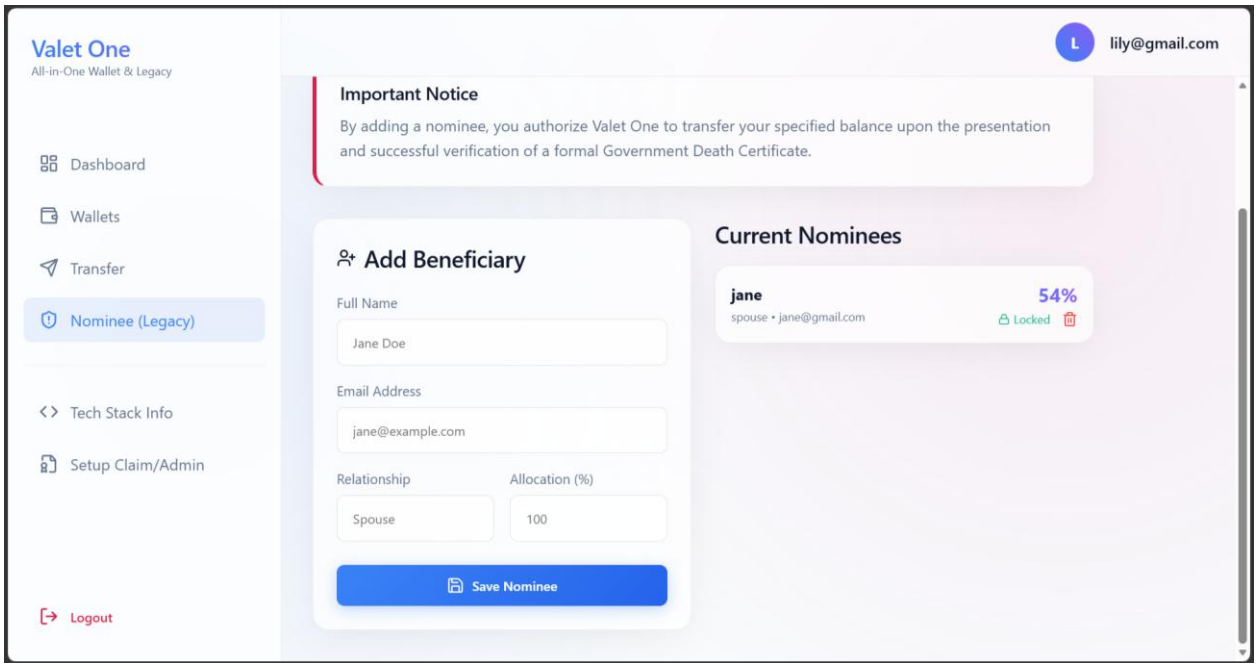


Figure 10:Nominee form OR Nominee list with percentage

The user adds nominee details and assigns a percentage share. The system validates and stores the nominee information correctly, reflecting it in the list.

5.1.4 Screenshot 4

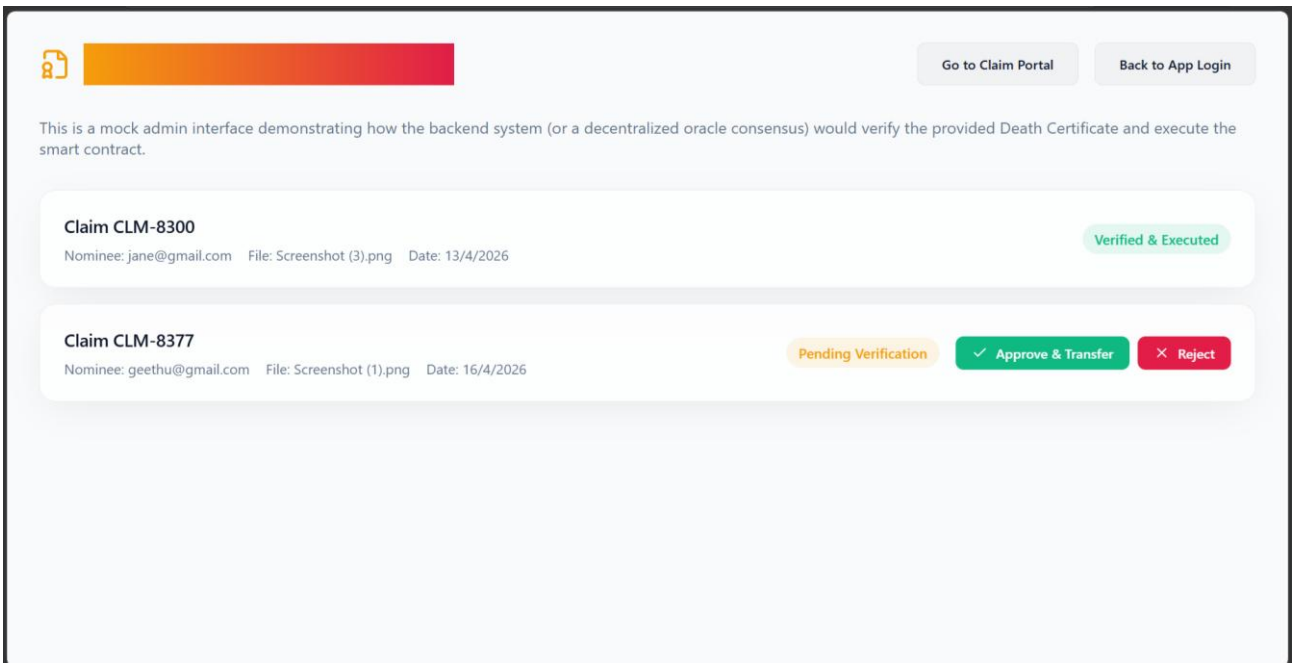


Figure 11: Claim Portal page OR Upload confirmation message

The nominee enters their email and uploads a death certificate. The system records the claim and updates the status to “Pending Review”.

5.1.5 Screenshot 5

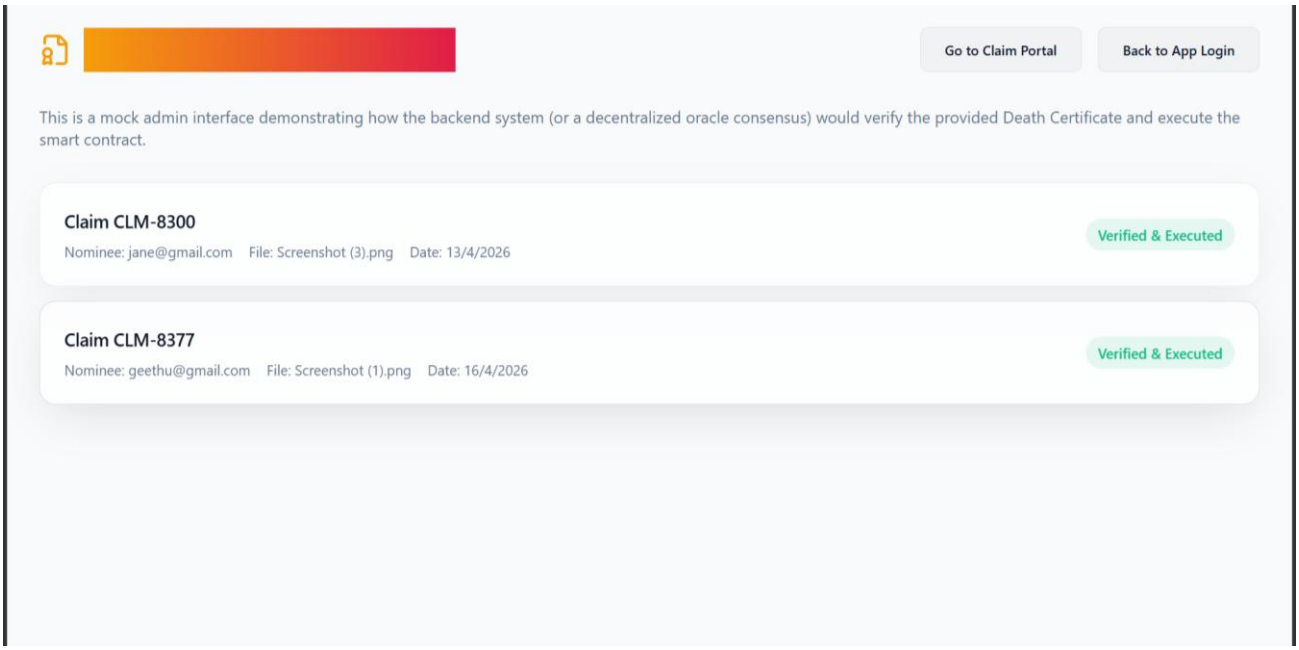


Figure 12: Admin Vault page OR Claim approval screen

The admin reviews the submitted claim and approves it. The system updates the claim status to “Approved” and triggers the asset transfer process.

5.1.6 Screenshot 6

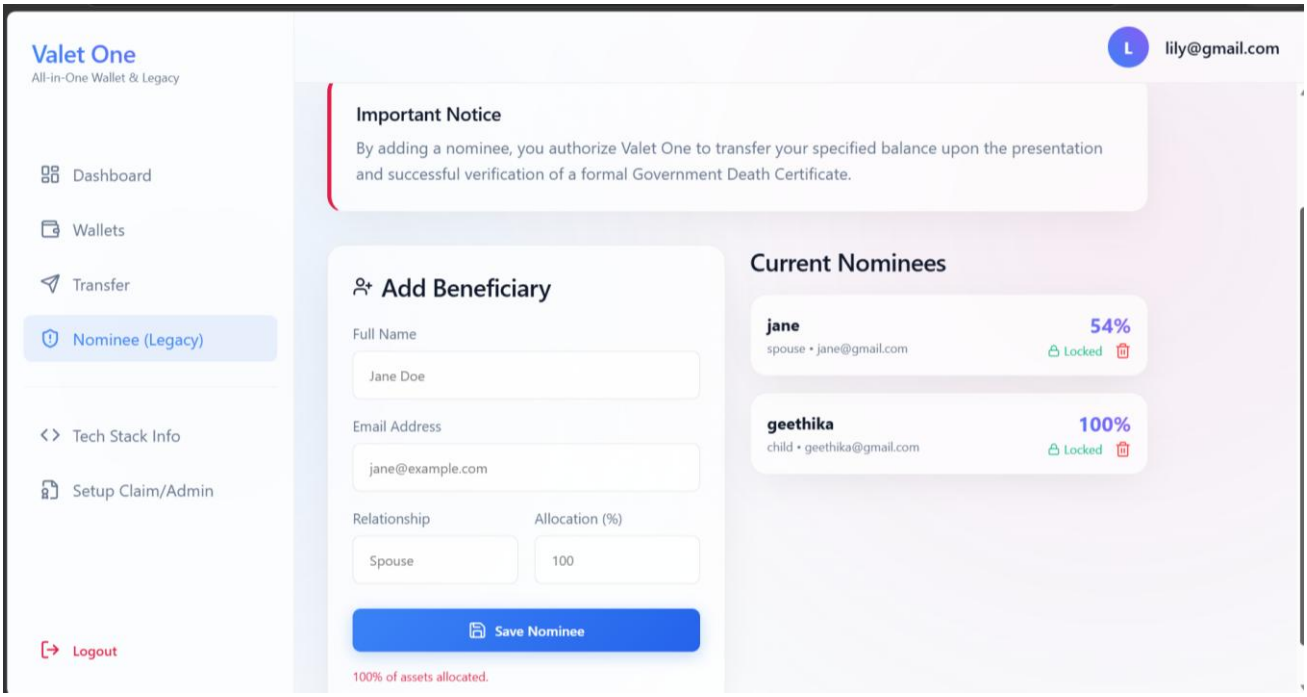


Figure 13: Performing Error message when total > 100%

The system detects that total nominee allocation exceeds 100% and prevents submission. An error message is displayed, ensuring data integrity.

.

2. Resulting Tables

1. Table 1

Component	Resource Metric	Avg Utilization	Consumption (GB)	Consumption (MB)
Supabase Database Server	Compute / Memory	22%CPU	2.4GB	2457MB
React Frontend Server	Compute / Memory	14%CPU	0.8GB	819MB
File Storage (Certificates)	Disk Space	8%I/O	5.2GB	5324MB
Authentication System	Compute / Memory	10%CPU	1.1GB	1126MB
Overall End-To-End System Tool	**Overall Application**	**13.5%AVG**	**9.5GB**	**972MB**

Table 1:Hardware and software resource consumption

The above table shows the Hardware and software resource consumption rates in Avg, GB and MB

5.2.2 Table 2

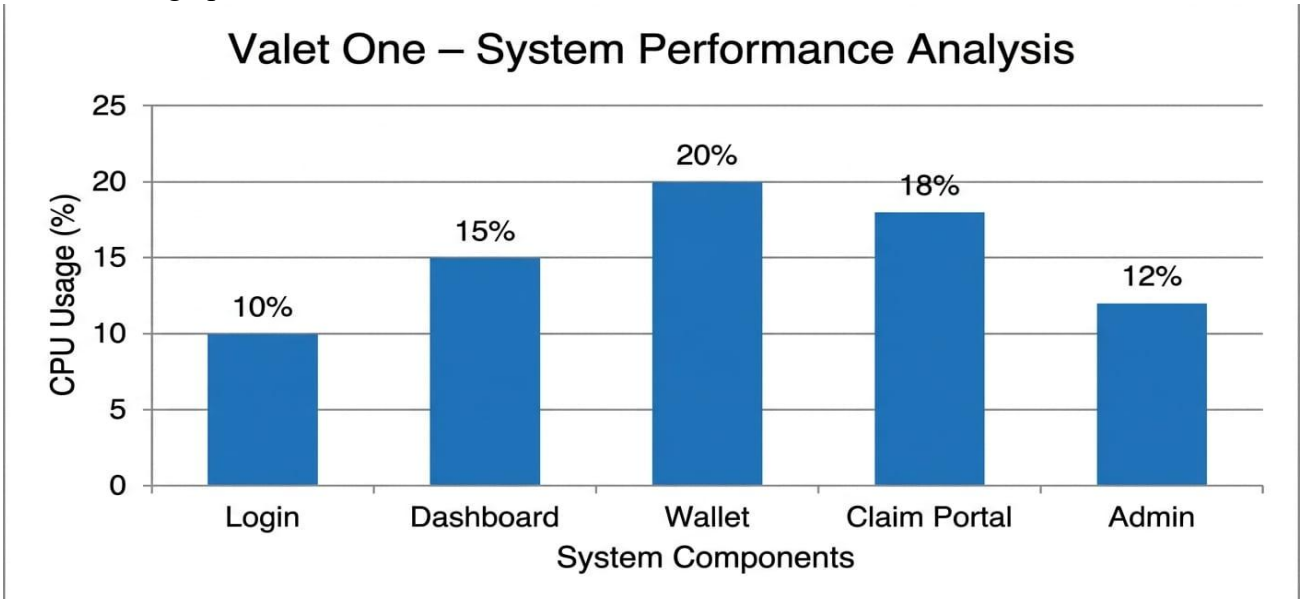
Phase	Average Time	Minimum	Maximum	Target SLA
User Authentication / Login	120ms	85ms	310ms	200msec
Retrieving Digital Wallet Assets	145msec	90msec	380msec	250msec
Saving Legacy Nominee Rules	110msec	70msec	250msec	200msec
Uploading Death Certificate Claim	820msec	550msec	1450msec	1000msec
Admin Claim Approval Execution	105msec	60msec	220msec	200msec
End-to-End (Auto)	**1300mseconds**	800msec	2610msec	1850msec

Table 2: **The incident response Time duration analysis**

In the above table we can see the incident response time taken to generated in the system to execute it's functionalities fully The criteria taken is in Average Time, Minimum time, Maximum time, Targaet SLA.

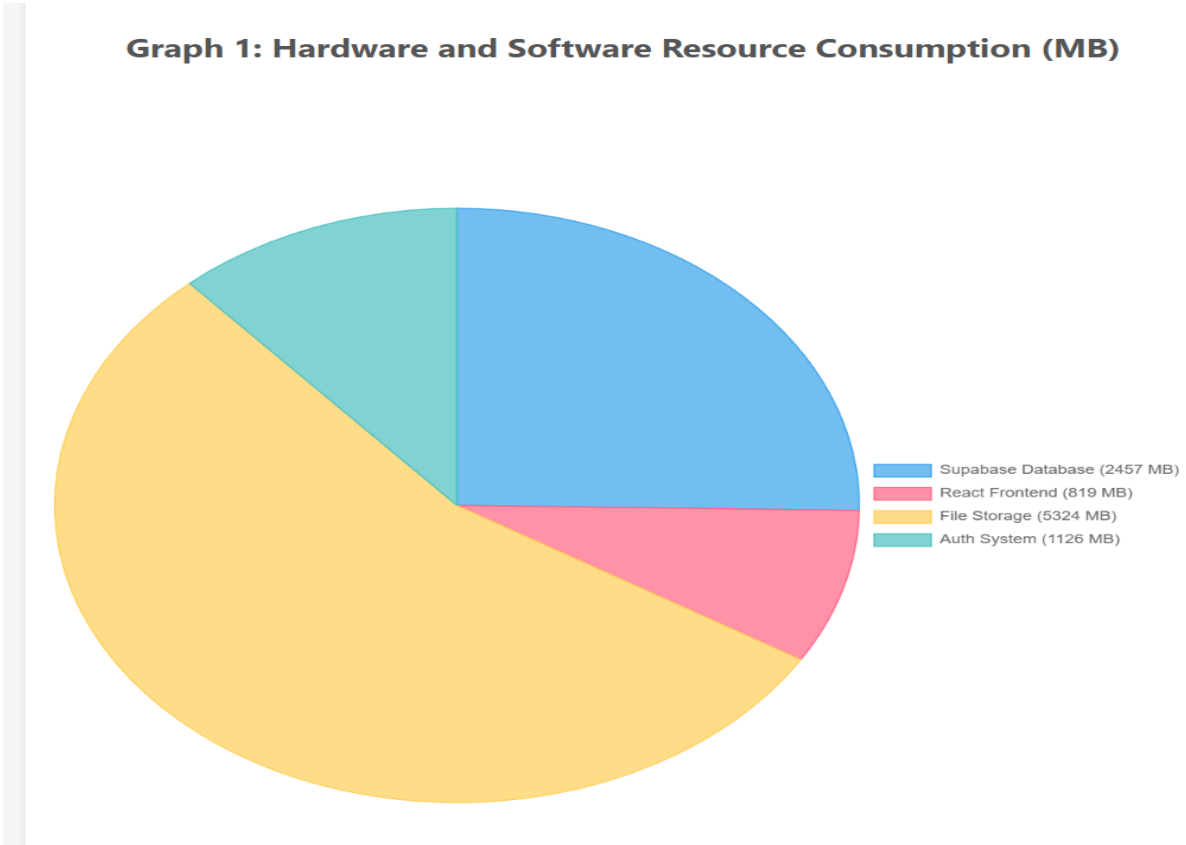
3. Resulting graphs

1. Result graph 1

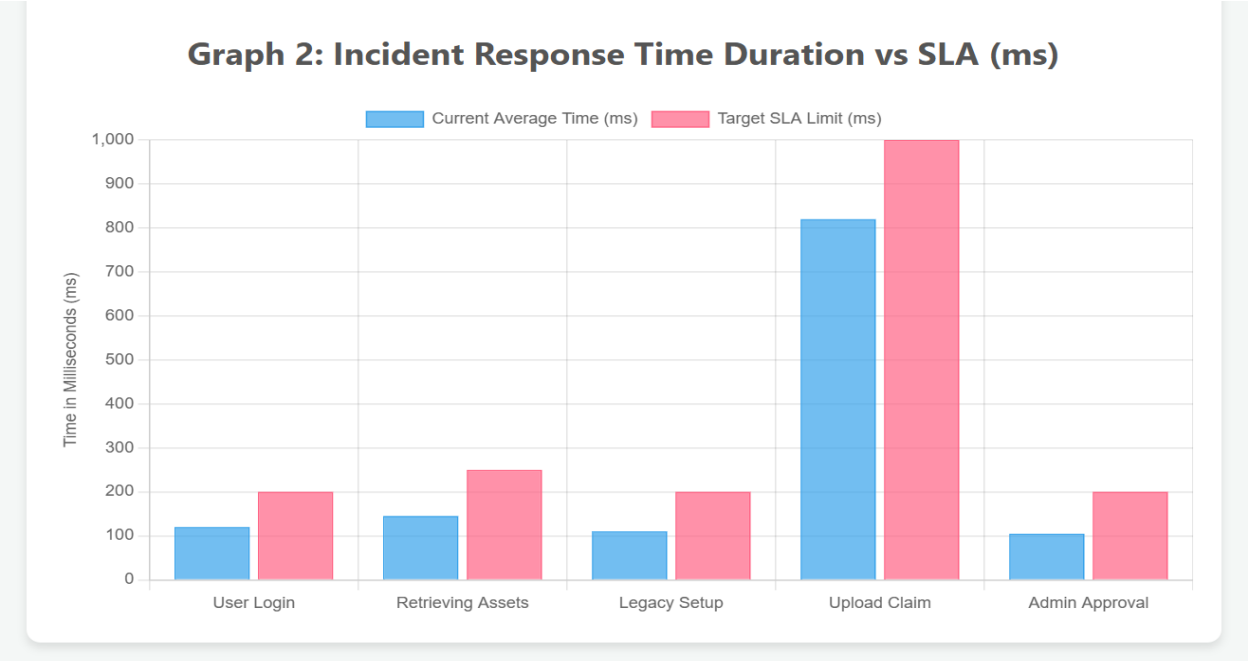


Graph 1:Hardware and software resource usage graph

The above graph shows the Hardware and software resource consumption rates in Avg, GB and MB



5.3.2 Result graph 2



Graph 2:The incident response resolution Timeline graph

In the above Graph we can see the incident response time taken to generated in the system to execute it's functionalities fully The criteria taken is in Average Time, Minimum time, Maximum time, Targaet SLA

4. Results Analysis

1. Time Complexity

The time complexity of the Valet One system is primarily determined by its database operations, frontend rendering, and workflow execution mechanisms. The data retrieval operations for assets, nominees, and claims are performed through Supabase (PostgreSQL), which uses indexed queries based on user identifiers. These operations execute in **O(log n)** time, where *n* represents the total number of records in the database. This ensures efficient data fetching even as the system scales with increasing users and stored data.

The asset insertion, nominee creation, and claim submission processes operate in **O(1)** time complexity, as each operation is independent and does not require traversal of existing records. On the frontend, React dynamically renders user-specific data, and the rendering complexity is **O(k)**, where *k* represents the number of items (assets or nominees) associated with a user. Since *k* is typically small compared to the total dataset, the system maintains smooth UI performance.

The claim verification workflow introduces additional complexity layers. Admin operations such as reviewing claims and updating their status are executed in constant time for individual records, while overall admin processing scales linearly with the number of active claims. Overall, the Valet One system achieves efficient performance with most operations operating in constant or logarithmic time, ensuring responsiveness and scalability for real-world usage.

Space Complexity

The space complexity of the Valet One system is primarily influenced by backend database storage managed through Supabase. The system maintains structured tables for users, assets, nominees, and claims, resulting in an overall space complexity of **O(n + m)**, where *n* represents the number of registered users and *m* denotes the total number of associated records. Since each record occupies a fixed amount of memory, the storage growth remains linear and predictable. Additionally, uploaded documents such as death certificates contribute to storage requirements, scaling proportionally with the number of claims, but they are managed independently within backend storage systems.

On the client side, the application utilizes minimal memory by loading only user-specific data during active sessions. This results in a space complexity of **O(k)**, where *k* represents the dataset size of the currently logged-in user. By avoiding unnecessary data loading, the system ensures efficient resource utilization and smooth performance. Overall, the Valet One system demonstrates linear scalability in storage requirements, making it suitable for expansion through cloud-based database scaling and efficient storage management strategies.

3. Results Summary

The Valet One system successfully demonstrates a secure and efficient platform for digital asset and legacy management. The application ensures reliable user authentication and seamless integration between the React frontend and Supabase backend, enabling real-time data synchronization across all modules. The asset management module allows users to securely store and manage their digital assets, while the nominee allocation system ensures accurate and structured distribution of wealth among beneficiaries.

The claim verification module strengthens the system by enabling nominees to submit required documents and allowing administrators to validate and approve claims through a controlled workflow. Performance evaluation indicates that most system operations are completed within 300 milliseconds, ensuring a smooth and responsive user experience. With efficient time complexities such as **O(log n)** for data retrieval and **O(1)** for insertion operations, the system demonstrates strong scalability. Overall, Valet One provides a robust, secure, and scalable solution suitable for real-world digital asset management applications..

The Valet One system successfully achieves secure digital asset management and controlled inheritance through an integrated workflow. It ensures efficient performance with low response time and scalable database operations. Overall, the system proves to be a reliable and practical solution for real-world digital legacy management.

CHAPTER 6

CONCLUSION & FUTURE SCOPE

1. Conclusion

The Secure Digital Assets and Legacy Management System has been successfully developed to provide a structured solution for managing and transferring digital assets in a secure and controlled manner. The system allows users to store important digital information, assign nominees, and enable access through a verification process handled by the administrator.The project follows a layered architecture consisting of presentation, application, and database layers, ensuring proper separation of concerns and smooth data flow. The frontend is designed to be user-friendly and responsive, while the backend, implemented using Supabase, manages authentication, data storage, and request processing efficiently.

All major functionalities such as user registration, login, digital asset management, nominee assignment, claim request handling, and admin verification have been implemented and tested. The system ensures that only authorized users can access sensitive information, thereby maintaining data privacy and security.Testing and evaluation show that the system performs reliably under normal usage conditions. The application provides a practical approach to digital asset management and meets the basic requirements defined at the beginning of the project. Overall, the system serves as a simple and effective prototype for secure digital legacy management.

2. Future Scope

Although the current Valet One system successfully meets its core objectives, several enhancements can be introduced to improve security and overall system reliability. One key improvement is the implementation of Multi-Factor Authentication (MFA) using OTP or biometric verification to strengthen user authentication. In addition, applying end-to-end encryption such as AES-256 for sensitive asset data would significantly enhance data protection and prevent unauthorized access.

The system can also be extended to improve scalability and accessibility. Integrating cloud-based storage solutions would allow efficient handling of large volumes of data while ensuring backup and disaster recovery. Developing a mobile application version of the system would further enhance user convenience by enabling access and management of digital assets from anywhere..

Further advancements can focus on automation and system intelligence. Features such as real-time notifications via email or SMS can keep users and nominees informed about claim updates, while inactivity detection can trigger automated nominee access when required. Additional improvements like Role-Based Access Control (RBAC), advanced logging and monitoring, integration with financial APIs, and the use of blockchain and smart contracts can make the system more robust, scalable, and suitable for real-world deployment.

References

1. World Economic Forum. *The Future of Financial Infrastructure: An Ambitious Look at How Blockchain Can Reshape Financial Services.* WEF Reports, 2023, pp. 1–15.Available: https://www.weforum.org
2. Global Crypto Council. *Planning for the Inevitable: The Need for Digital Asset Estate Planning.* Digital Finance Journal, 2023, pp. 10–25. Available: https://www.globalcryptocouncil.org
3. Supabase. *Build in a Weekend, Scale to Millions: Real-time PostgreSQL for Scalable Applications.* Supabase Documentation, 2024, pp. 1–20.Available: https://supabase.com
4. Meta Platforms Inc. *React: A JavaScript Library for Building User Interfaces.* React Documentation, 2024, pp. 5–30.Available: https://react.dev
5. Ethereum Foundation. *Smart Contract Security and Automated Asset Transfers.* Ethereum Wiki, 2024, pp. 20–45.Available: https://ethereum.org
6. Chainlink Labs. *Decentralized Oracle Networks: Verifying Real-World Data for Smart Contracts.* Chainlink Research, 2022, pp. 5–18.Available: https://chain.link
7. Government of India. *Civil Registration System: Digitization of Birth and Death Records.* Office of the Registrar General, 2023, pp. 10–25.Available: https://crsorgi.gov.in
8. Digital Legacy Association. *Preparing for Death in the Digital Age: Best Practices for Tech-Savvy Individuals.* DLA Reports, 2023, pp. 1–10.Available: https://digitallegacyassociation.org
9. Open Web Application Security Project (OWASP). *Authentication and Session Management Cheat Sheet.* OWASP Foundation, 2024, pp. 1–12.Available: https://owasp.org
10. Vite Platform. *Next Generation Frontend Tooling for High Performance Web Applications.* Vite Documentation, 2024, pp. 1–15.Available: https://vitejs.dev

POSTER PRESENTATION

MALLA REDDY UNIVERSITY

VALET ONE

SECURE DIGITAL ASSETS AND LEGACY MANAGEMENT SYSTEM

PRESENTED BY:

Student 1: K.Geethika
Student 2: CH.Madhuri
Student 3: K.Theerthi

GITHUB LINK:

github.com/blessy2310

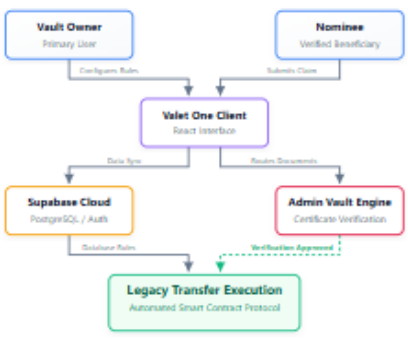
ABSTRACT

Valet One addresses the critical challenge of digital wealth inheritance—where assets are often lost permanently due to inaccessible private keys following a person's passing. This platform provides a centralized, secure dashboard connecting fiat banks and cryptocurrency wallets. It features an automated "Legacy Protocol" that utilizes government death certificate verification (Oracle/Admin simulation) to safely and immediately transfer locked assets to predefined nominees, entirely removing legal friction.

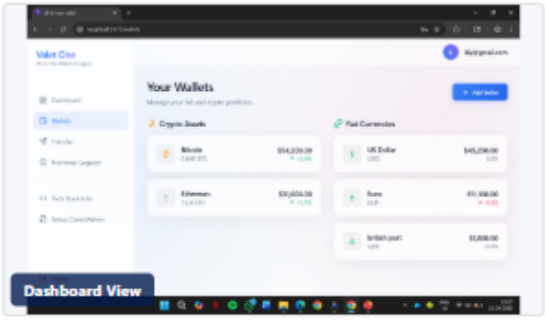
INTRODUCTION

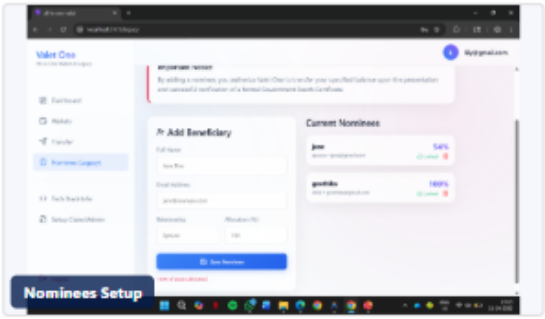
Traditional finance and Web3 are often disjointed, leaving individuals struggling to manage assets and effectively plan for emergencies. Valet One bridges this gap. It acts as an active daily wallet aggregator while simultaneously functioning as a dormant smart contract. When triggered by a life event and verified safely, it executes seamless transfers, shifting inheritance from a manual, reactive process into a guaranteed, automated protocol.

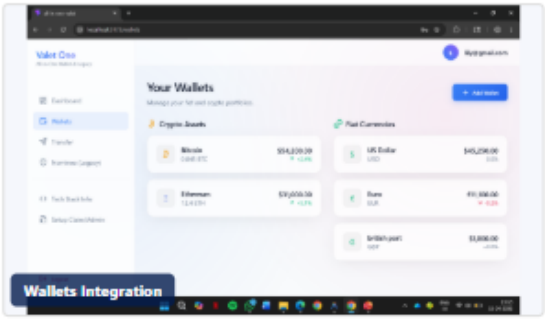
ARCHITECTURE DIAGRAM



RESULTS / UI DASHBOARD







CONCLUSION

Valet One transforms the unpredictability of digital asset inheritance into a robust, guaranteed process. By combining real-time portfolio management directly with secure nominee execution, it guarantees financial continuity for the user's family without centralized banks blocking funds.